การทดสอบ ETL Pipeline ด้วย Apache Airflow และ pandas

เพื่อให้การเปรียบเทียบมีความเที่ยงตรง การทดสอบ Airflow + pandas ใช้โครงสร้างเดียวกับ Airflow + PySpark ทุกประการ ไม่ว่าจะเป็น Airflow 2.8.0, DAG Structure, ขั้นตอน Transform , Dataset ทั้ง 3 ขนาด และ Source/Target Database เดิม โดยมีสิ่งที่เปลี่ยนแปลงเพียง 3 อย่างเท่านั้น

1. **Processing Engine** : เปลี่ยนจาก PySpark เป็น pandas แทน
2. **Load Method** : เปลี่ยนจาก PySpark JDBC Writer เป็น pymysql
3. **Target Table** : สร้างตาราง `fraud_clean_airflow_pandas` ใน MySQL (fraud_dl) แยกออกมา โดยใช้ Schema เดียวกับ `fraud_clean_airflow` ทุกประการ เพื่อให้สามารถเปรียบเทียบผลลัพธ์ระหว่างสอง Engine ได้โดยตรง

สามารถทำตามขั้นตอนได้ ดังนี้

STEP 1 : ติดตั้ง Library เพิ่มเติม

เช็คก่อนรันว่า activate แล้ว โดยต้องเห็น `(airflow_env)` นำหน้าอยู่:

```
source ~/airflow_env/bin/activate
pip install pandas
```

ตรวจสอบหลังติดตั้งเสร็จ:

```
pip list | grep -E 'pandas'
```

STEP 2 : เพิ่ม requirements.txt

```
pip list --format=freeze | grep -E 'pandas' >> airflow_env/
requirements.txt
```

STEP 3 : สร้าง Table ใหม่ใน MySQL

```
USE fraud_dl;
CREATE TABLE IF NOT EXISTS fraud_clean_airflow_pandas (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    trans_datetime DATETIME,
    cc_num_hash     VARCHAR(64),
    merchant        VARCHAR(150),
    category        VARCHAR(50),
    amt             DECIMAL(10,2),
    first_hash      VARCHAR(64),
    last_hash       VARCHAR(64),
    gender          CHAR(50),
    city            VARCHAR(50),
    state           CHAR(50),
    city_pop        INT,
    job             VARCHAR(100),
    dob_hash        VARCHAR(64),
    trans_num       VARCHAR(50),
    is_fraud        TINYINT(1),
    loaded_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

STEP 4 : สร้างไฟล์ DAG

เปิด VS Code สร้างไฟล์ `fraud_etl_pandas.py` ในโฟลเดอร์ dags วางโค้ดนี้:

```
# ============================================================
====================
# fraud_etl_pandas.py
# Apache Airflow 2.8.0 + pandas ETL Pipeline
# Credit Card Fraud Detection Dataset
# ============================================================
====================
import os
import csv
import time
import hashlib
```

```
import pyodbc
import pandas as pd
import pymysql
from datetime import datetime
from dotenv import load_dotenv

from airflow import DAG
from airflow.operators.python import PythonOperator

# ============================================================
====================
# โหลด .env
# ============================================================
====================
load_dotenv(os.path.join(os.path.dirname(__file__), '.en
v'))

# ============================================================
====================
# Config
# ============================================================
====================
MSSQL_CONN = (
    f"DRIVER={{ODBC Driver 17 for SQL Server}};"
    f"SERVER={os.getenv('MSSQL_SERVER')};"
    f"DATABASE={os.getenv('MSSQL_DATABASE')};"
    f"UID={os.getenv('MSSQL_UID')};"
    f"PWD={os.getenv('MSSQL_PWD')}"
)

PARQUET_DIR = "/tmp/fraud_etl"
CSV_LOG     = "/tmp/fraud_etl/timing_log_airflow_pandas.cs
v"
DATASETS    = {"10k": "fraud_10k", "100k": "fraud_100k", "1
m": "fraud_1m"}

# ============================================================
====================
```

```
# TASK 1 — EXTRACT
# ============================================================
====================
def extract(dataset_key: str, **kwargs):
    ti       = kwargs['ti']
    table    = DATASETS[dataset_key]
    out_path = f'{PARQUET_DIR}/pandas_raw_{dataset_key}.par
quet'

    t_start = time.perf_counter()

    conn = pyodbc.connect(MSSQL_CONN)
    df   = pd.read_sql(f'SELECT * FROM {table}', conn)
    conn.close()

    df.to_parquet(out_path, index=False)

    extract_sec = time.perf_counter() - t_start

    ti.xcom_push(key=f'extract_sec_{dataset_key}', value=ex
tract_sec)
    ti.xcom_push(key=f'parquet_raw_{dataset_key}', value=ou
t_path)

    print(f'[Extract] {dataset_key}: {len(df)} rows | {extr
act_sec:.3f}s')

# ============================================================
====================
# TASK 2 — TRANSFORM
# ============================================================
====================
def transform(dataset_key: str, **kwargs):
    ti       = kwargs['ti']
    in_path  = ti.xcom_pull(key=f'parquet_raw_{dataset_ke
y}')
    out_path = f'{PARQUET_DIR}/pandas_clean_{dataset_key}.p
arquet'
```

```
    t_start = time.perf_counter()

    df = pd.read_parquet(in_path)

    # Step 1: Hash ข้อมูลส่วนตัว
    def sha256(x):
        return hashlib.sha256(str(x).encode()).hexdigest()
if pd.notna(x) else None

    df['cc_num_hash'] = df['cc_num'].apply(sha256)
    df['first_hash']  = df['first'].apply(sha256)
    df['last_hash']   = df['last'].apply(sha256)
    df['dob_hash']    = df['dob'].apply(sha256)

    # Step 2: แปลง DateTime
    df['trans_datetime'] = pd.to_datetime(df['trans_date_tr
ans_time'])

    # Step 3: Filter
    df = df[df['amt'].notna()]
    df = df[df['amt'] > 0]

    # Step 4: Select เฉพาะ column ที่ต้องการ
    df = df[[
        'trans_datetime', 'cc_num_hash', 'merchant', 'categ
ory',
        'amt', 'first_hash', 'last_hash', 'gender', 'city',
'state',
        'city_pop', 'job', 'dob_hash', 'trans_num', 'is_fra
ud'
    ]]

    df.to_parquet(out_path, index=False)

    transform_sec = time.perf_counter() - t_start

    ti.xcom_push(key=f'transform_sec_{dataset_key}', value=
```

```
transform_sec)
    ti.xcom_push(key=f'parquet_clean_{dataset_key}', value=
out_path)

    print(f'[Transform] {dataset_key}: {transform_sec:.3f}
s')

# ============================================================
====================
# TASK 3 — LOAD
# ============================================================
====================
def load(dataset_key: str, **kwargs):
    ti      = kwargs['ti']
    in_path = ti.xcom_pull(key=f'parquet_clean_{dataset_ke
y}')

    t_start = time.perf_counter()

    df = pd.read_parquet(in_path)

    conn = pymysql.connect(
        host=os.getenv('MYSQL_HOST'),
        port=int(os.getenv('MYSQL_PORT', '3306')),
        user=os.getenv('MYSQL_USER'),
        password=os.getenv('MYSQL_PWD'),
        database='fraud_dl'
    )
    cursor = conn.cursor()

    cols         = ','.join(df.columns)
    placeholders = ','.join(['%s'] * len(df.columns))
    sql          = f"INSERT INTO fraud_clean_airflow_pandas
({cols}) VALUES ({placeholders})"

    batch_size = 1000
    rows = [tuple(row) for row in df.itertuples(index=Fals
e, name=None)]
```

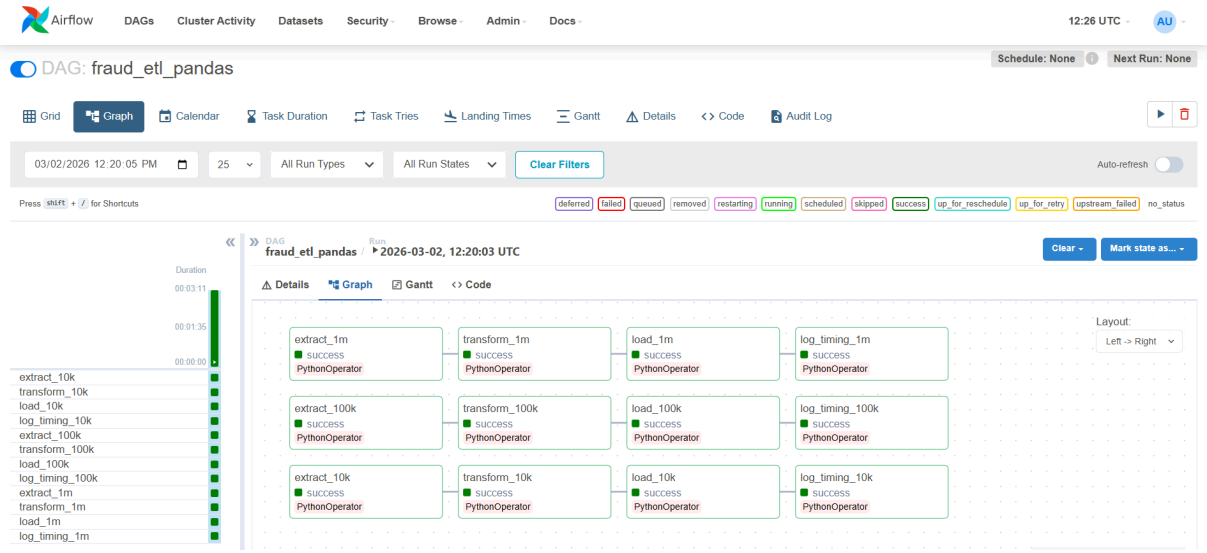
```
    for i in range(0, len(rows), batch_size):
        cursor.executemany(sql, rows[i:i+batch_size])
        conn.commit()

    cursor.close()
    conn.close()

    load_sec = time.perf_counter() - t_start

    ti.xcom_push(key=f'load_sec_{dataset_key}', value=load_
sec)
    print(f'[Load] {dataset_key}: {load_sec:.3f}s')

# ============================================================
====================
# TASK 4 — LOG TIMING
# ============================================================
====================
def log_timing(dataset_key: str, **kwargs):
    ti = kwargs['ti']

    extract_sec   = ti.xcom_pull(key=f'extract_sec_{dataset
_key}')
    transform_sec = ti.xcom_pull(key=f'transform_sec_{datas
et_key}')
    load_sec      = ti.xcom_pull(key=f'load_sec_{dataset_ke
y}')

    write_header = not os.path.exists(CSV_LOG)
    with open(CSV_LOG, 'a', newline='') as f:
        writer = csv.writer(f)
        if write_header:
            writer.writerow(['dataset', 'extract_sec', 'tra
nsform_sec', 'load_sec'])
        writer.writerow([dataset_key,
                         round(extract_sec,   4),
                         round(transform_sec, 4),
                         round(load_sec,      4)])
```

```
    print(f'[Log] {dataset_key}: extract={extract_sec:.3f}s
| transform={transform_sec:.3f}s | load={load_sec:.3f}s')

# ============================================================
====================
# DAG Definition
# ============================================================
====================
default_args = {'owner': 'airflow', 'retries': 0}

with DAG(
    dag_id='fraud_etl_pandas',
    default_args=default_args,
    schedule_interval=None,
    start_date=datetime(2024, 1, 1),
    catchup=False
) as dag:

    for key in ['10k', '100k', '1m']:
        extract_task = PythonOperator(
            task_id=f'extract_{key}',
            python_callable=extract,
            op_kwargs={'dataset_key': key}
        )
        transform_task = PythonOperator(
            task_id=f'transform_{key}',
            python_callable=transform,
            op_kwargs={'dataset_key': key}
        )
        load_task = PythonOperator(
            task_id=f'load_{key}',
            python_callable=load,
            op_kwargs={'dataset_key': key}
        )
        log_timing_task = PythonOperator(
            task_id=f'log_timing_{key}',
            python_callable=log_timing,
```

```
            op_kwargs={'dataset_key': key}
        )

        extract_task >> transform_task >> load_task >> log_
timing_task
```

STEP 5 : Trigger DAG

กลับไปที่ `http://localhost:8080` แล้ว Trigger DAG



STEP 6 : ตรวจสอบข้อมูลใน MySQL

รัน SQL ใน MySQL :

```
USE fraud_dl;

-- นับแถวทั้งหมด
SELECT COUNT(*) AS total_rows FROM fraud_clean_airflow_pand
as;

-- ดูตัวอย่างข้อมูล
SELECT * FROM fraud_clean_airflow_pandas LIMIT 5;

-- ตรวจว่า hash ถูกต้อง
SELECT
```

```
    cc_hash,
    first_hash,
    last_hash,
    dob_hash
FROM fraud_clean_airflow_pandas LIMIT 5;

-- ตรวจ amt ว่าไม่มี NULL หรือ <= 0
SELECT COUNT(*) AS bad_amt
FROM fraud_clean_airflow_pandas
WHERE amt IS NULL OR amt <= 0;
```

STEP 7 : ตรวจสอบผลลัพธ์

ดู CSV หลังรันเสร็จ:

```
cat /tmp/fraud_etl/timing_log_airflow_pandas.csv
```